

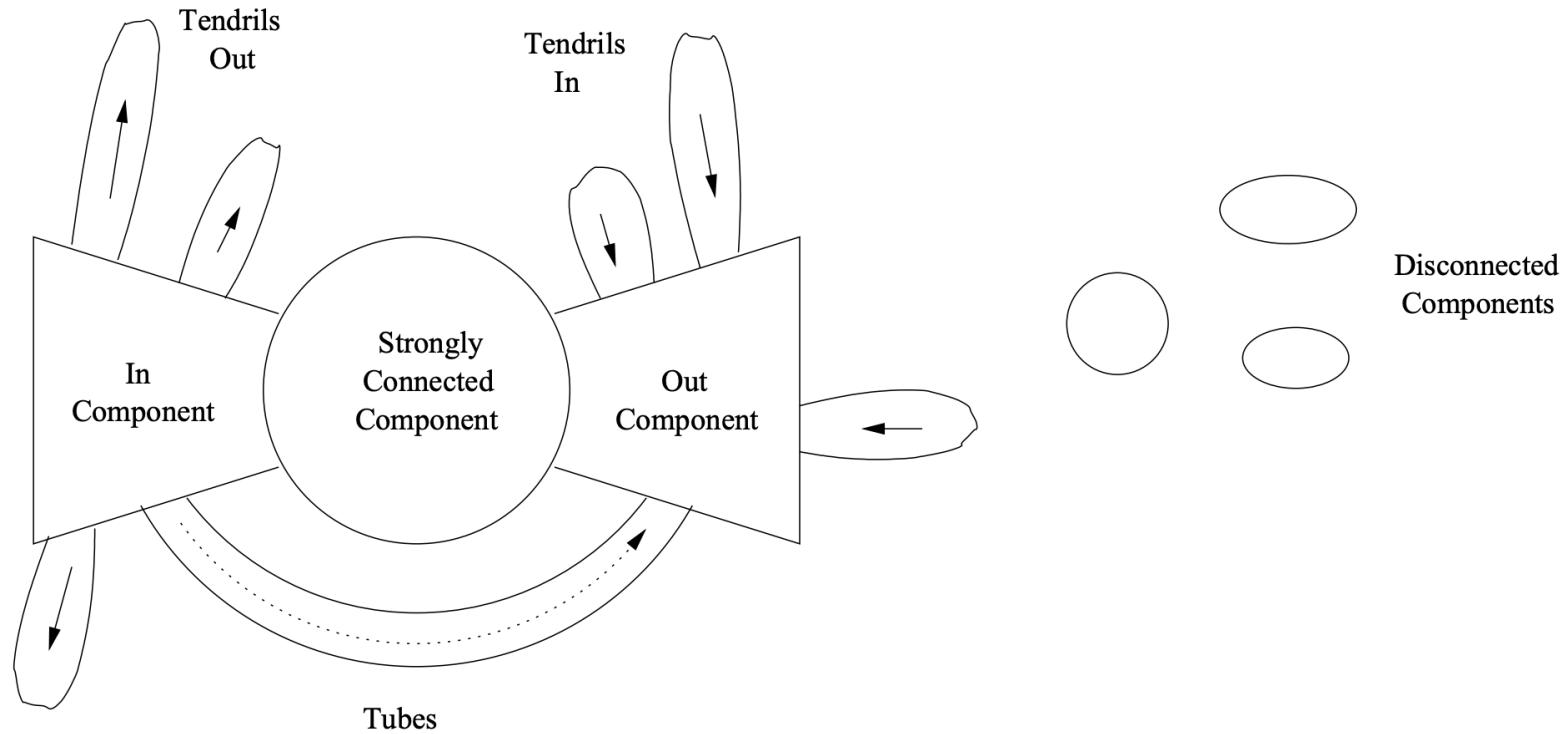
Algorithms for Large Graphs

COMPSCI 753

Ruofan Wang

Slides adapted from “Mining of Massive Datasets” <http://mmds.org>

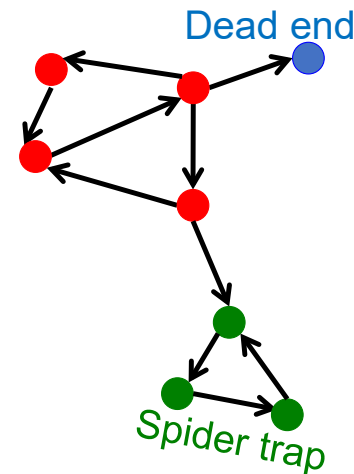
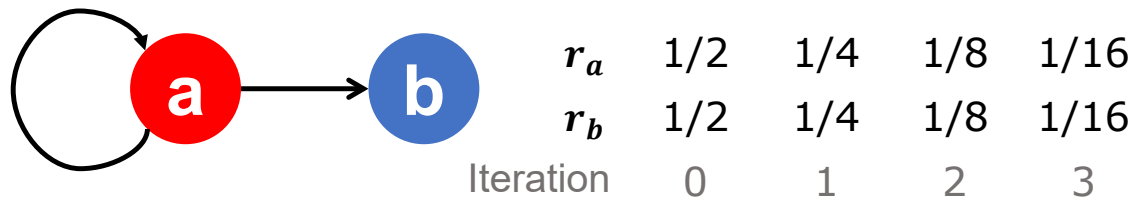
Structure of Web



Problems of PageRank

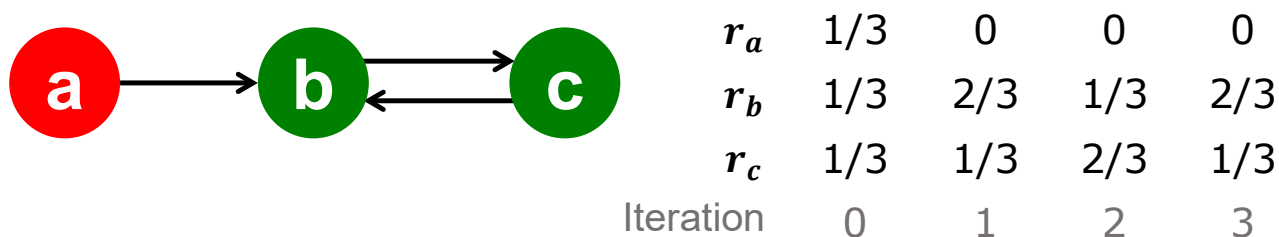
■ Problem 1: Dead ends

- Pages that do not have out-links
- Information propagates to dead ends will leak out



■ Problem 2: Spider traps

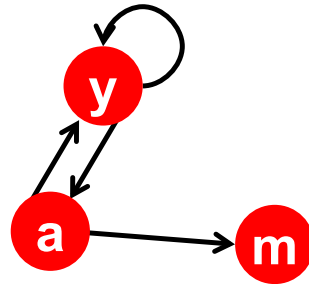
- Random walk gets “stuck” in a trap
- And eventually spider traps absorb all importance



$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

Dead ends

- How to solve the problem of **dead ends**?
 - Simple idea – Run PageRank in the subgraph without dead ends and then propagate the ranks to the dead ends.



r_y	1/2	3/4	...	2/3	1/2
r_a	1/2	1/4	...	1/3	1/3
r_m	-	-	-	0	1/6

Iteration 0 1 stop

G

	y	a	m
y	1/2	1/2	0
a	1/2	0	0
m	0	1/2	0

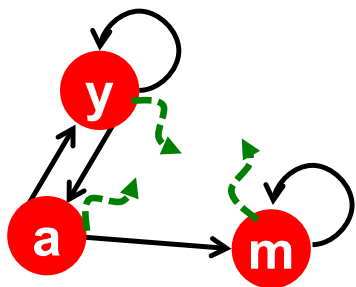
Not column stochastic!

G'

	y	a
y	1/2	1
a	1/2	0

Spider trap

- Solution to **spider trap** (assume no dead end exists):
 - Idea – Get out of the trap by randomly select a node to teleport.
 - For each page, the random surfer has two choices:
 - Follow the outlinks to propagate information (with probability β)
 - **Randomly jump** to a node (with probability $1 - \beta$)



Flow form

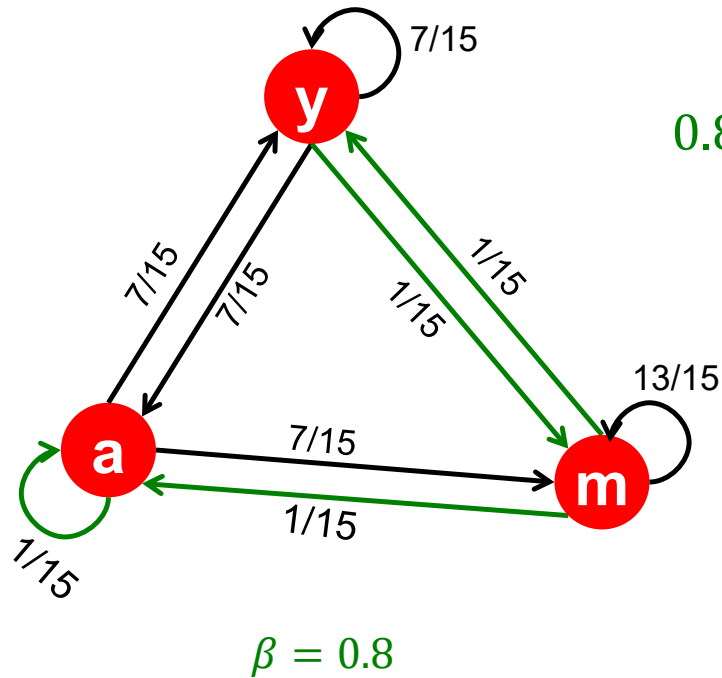
$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + \sum_i (1 - \beta) \frac{1}{N} r_i$$

Matrix form for power iteration

$$\mathbf{r} = \left[\beta \mathbf{M} + (1 - \beta) \left[\frac{1}{N} \right]_{N \times N} \right] \mathbf{r}$$

$\mathbf{A}$

Teleport example for spider trap



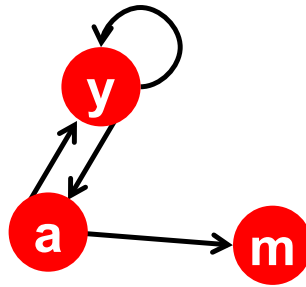
$$\begin{array}{c}
 \mathbf{M} \\
 \begin{array}{c} \mathbf{y} \\ \mathbf{a} \\ \mathbf{m} \end{array}
 \end{array}
 \begin{array}{c|c|c}
 \mathbf{y} & \mathbf{a} & \mathbf{m} \\
 \hline
 \mathbf{y} & 1/2 & 1/2 & 0 \\
 \mathbf{a} & 1/2 & 0 & 0 \\
 \mathbf{m} & 0 & 1/2 & 1
 \end{array}
 + 0.2 \times
 \begin{array}{c}
 \mathbf{y} \\ \mathbf{a} \\ \mathbf{m}
 \end{array}
 \begin{array}{c|c|c}
 \mathbf{y} & \mathbf{a} & \mathbf{m} \\
 \hline
 \mathbf{y} & 1/3 & 1/3 & 1/3 \\
 \mathbf{a} & 1/3 & 1/3 & 1/3 \\
 \mathbf{m} & 1/3 & 1/3 & 1/3
 \end{array}$$

A

$$\begin{array}{c} \mathbf{y} \\ \mathbf{a} \\ \mathbf{m} \end{array}
 \begin{array}{c|c|c}
 \mathbf{y} & \mathbf{a} & \mathbf{m} \\
 \hline
 \mathbf{y} & 7/15 & 7/15 & 1/15 \\
 \mathbf{a} & 7/15 & 1/15 & 1/15 \\
 \mathbf{m} & 1/15 & 7/15 & 13/15
 \end{array}$$

Solving both problems with teleport

- Solving **dead ends** with teleport:
 - Idea – Always teleport at dead end nodes.
 - Adjust the graph and link the dead ends to all other nodes with equal prob. Let the new transition matrix be M'
 - Calculate $A = \beta M' + (1 - \beta) \left[\frac{1}{N} \right]_{N \times N}$



	y	a	m
y	1/2	1/2	1/3
a	1/2	0	1/3
m	0	1/2	1/3

M' may have dense columns

Efficiency of Power Iteration

- Matrix multiplication for many times!

- $\mathbf{r} = \mathbf{A} \cdot \mathbf{r}$

- Time cost: $O(N^2 I)$

- Space cost: $O(N^2)$

$$\begin{array}{c} \mathbf{A} \end{array} \begin{array}{c|c|c|c} & \mathbf{y} & \mathbf{a} & \mathbf{m} \\ \hline \mathbf{y} & 7/15 & 7/15 & 1/15 \\ \mathbf{a} & 7/15 & 1/15 & 1/15 \\ \mathbf{m} & 1/15 & 7/15 & 13/15 \end{array} = 0.8 \times \begin{array}{c} \mathbf{M} \end{array} \begin{array}{c|c|c|c} & \mathbf{y} & \mathbf{a} & \mathbf{m} \\ \hline \mathbf{y} & 1/2 & 1/2 & 0 \\ \mathbf{a} & 1/2 & 0 & 0 \\ \mathbf{m} & 0 & 1/2 & 1 \end{array} + 0.2 \times \begin{array}{c} \end{array} \begin{array}{c|c|c|c} & \mathbf{y} & \mathbf{a} & \mathbf{m} \\ \hline \mathbf{y} & 1/3 & 1/3 & 1/3 \\ \mathbf{a} & 1/3 & 1/3 & 1/3 \\ \mathbf{m} & 1/3 & 1/3 & 1/3 \end{array}$$

- How if we have $N = 1$ billion webpages?

- We will have $N^2 = 10^{18}$ entries in the matrix

Sparse matrix formulation

- We can rearrange the PageRank equation to the follow:

$$\mathbf{r} = \beta \mathbf{M} \mathbf{r} + (1 - \beta) \begin{bmatrix} 1/N & \cdots & 1/N \\ \vdots & \ddots & \vdots \\ 1/N & \cdots & 1/N \end{bmatrix} \begin{bmatrix} r_1 \\ \vdots \\ r_N \end{bmatrix} \rightarrow \mathbf{r} = \beta \mathbf{M} \mathbf{r} + \left[\frac{1 - \beta}{N} \right]_{N \times 1}$$

Constant for each row!

- M is sparse!

- In each iteration (suppose no dead-ends):

- Calculate $r_j^{new} = \beta \sum_{i \rightarrow j} \frac{r_i^{old}}{d_i}$
- Update the rank with the constant

$$r_j^{new} = r_j^{new} + \frac{1 - \beta}{N}$$

- Question: how about if we have dead-ends?

$$1. r_j^{new} = \beta \sum_{i \rightarrow j} \frac{r_i^{old}}{d_i}$$

$$2. S = \sum_j r_j^{new}$$

$$3. r_j^{new} = r_j^{new} + \frac{1-S}{N}$$

M	y	a	m
y	1/2	1/2	0
a	1/2	0	0
m	0	1/2	0

Limitations of PageRank

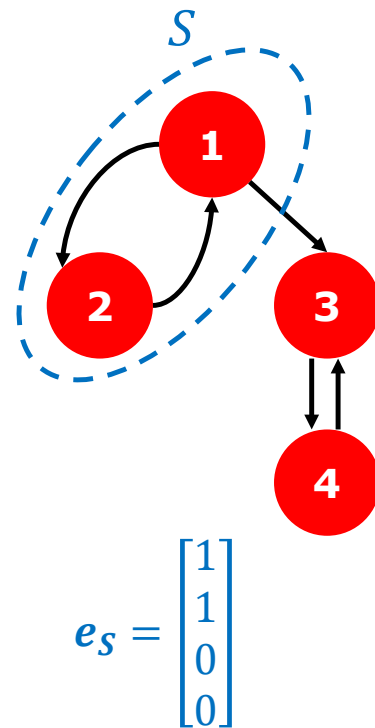
- Measures generic importance of a page
 - In different topics, the importance of a web page could be different!
- The results are not personalized
 - All the users will get the same results
- Web spam
 - Spammer can create some topological structures to boost PageRank

Biased PageRank

- In standard PageRank, every node is in the teleport set.
- In **biased PageRank**, teleport only to nodes within a set S .
- The matrix formulation of biased PageRank is

$$\mathbf{r} = \beta \mathbf{M} \mathbf{r} + (1 - \beta) \frac{1}{|S|} \mathbf{e}_S$$

$\mathbf{e}_S$ is a vector with the i -th value equals 1 if node i is in S , otherwise 0



Applications of Biased PageRank

- Topic-specific ranking
- Personalized PageRank
- TrustRank

Topic-specific ranking

- Intuition:
 - Under different contexts, the search engine should be able to return different ranks to a particular topic, e.g., “technology”.
 - E.g., a query of “apple” should be related to “technology” or “fruits”.
- Question: How to get ranking score within a topic?
 - **Teleport set** S_t = A set of pages relevant to topic t
 - For each teleport set S_t , we get a different PageRank r_t

$$r_t = \beta M r_t + (1 - \beta) \frac{1}{|S_t|} e_{S_t}$$

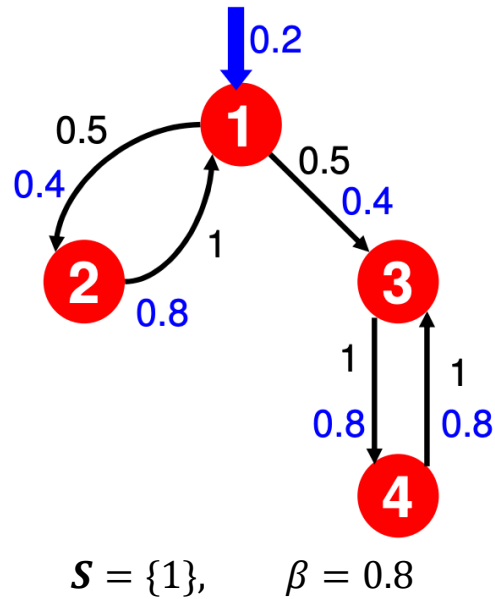
Topic-specific ranking

- Topic-specific PageRank:
 - For each topic t , we get PageRank r_t based on teleport set S_t
 - For each **query** q , classify it and get probability $p(t|q)$
 - Could use topic modeling or clustering
 - Final importance score obtained as linear combination

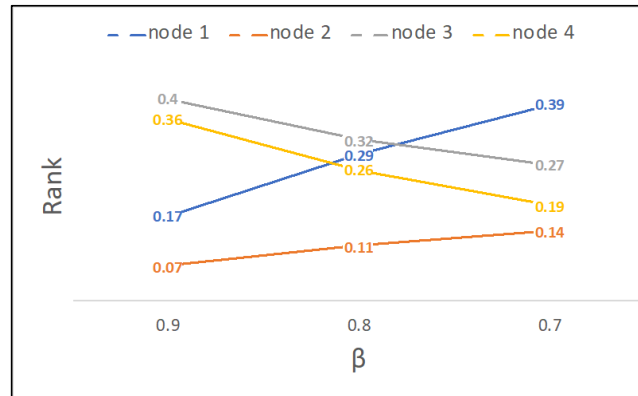
$$r = \sum_t p(t|q) r_t$$

Topic-specific ranking

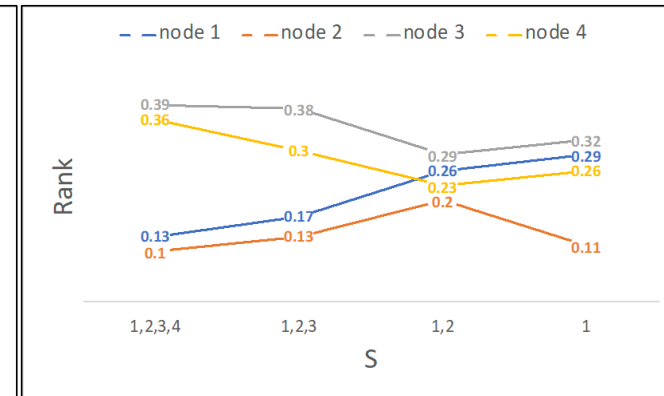
Example:



Node	Iter=0	Iter=1	Iter=2	...	Stop
1	0.25	0.4	0.28	...	0.294
2	0.25	0.1	0.16	...	0.118
3	0.25	0.3	0.32	...	0.327
4	0.25	0.2	0.24	...	0.261



Rank v.s. β



Rank v.s. S

Personalized PageRank

- **Idea:** Provide a personalized ranking for each user based on his/her preference on web pages S_u (e.g., browse history)

$$r_u = \beta M r_u + (1 - \beta) p_u$$

- $p_u = \frac{1}{|S_u|} e_{S_u}$ is called preference vector
- **Problems**
 - Computation for each personalized ranking r_u is expensive
 - Google has 2 billion users – **it is not possible to calculate a ranking based on each user!**

Calculating Personalized PageRank

- Basis vector: A one-hot vector where only one of the row is 1 and all others are 0.
- Each basis vector $\mathbf{e}_i$ denotes a teleport set of exactly one web page i (could use only representative webpages from different topics)
- Any user preference vector can be computed as a weighted sum of the basis vectors:

$$\mathbf{p}_u = \frac{1}{\sum_i w_i} \sum_{i \in S_u} w_i \mathbf{e}_i \quad \text{for } w_i \geq 0$$

- The Linearity Theorem says [Jeh and Widom '03]:

$$\mathbf{r}_u = \frac{1}{\sum_i w_i} \sum_{i \in S_u} w_i \mathbf{r}_i$$

Example

- Basis vectors:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad \mathbf{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

- Biased PageRank:

$$\mathbf{r}_1 = \begin{bmatrix} 0.1 \\ 0.7 \\ 0.2 \end{bmatrix} \quad \mathbf{r}_2 = \begin{bmatrix} 0.3 \\ 0.3 \\ 0.4 \end{bmatrix} \quad \mathbf{r}_3 = \begin{bmatrix} 0.2 \\ 0.1 \\ 0.7 \end{bmatrix}$$

User 1 – Teleport set $\mathbf{s}_{u1} = \{1,2\}$

$$\mathbf{p}_{u1} = \frac{1}{2} \mathbf{e}_{\mathbf{s}_{u1}} = \frac{1}{2} \mathbf{e}_1 + \frac{1}{2} \mathbf{e}_2$$

$$\mathbf{r}_{u1} = \frac{1}{2} \mathbf{r}_1 + \frac{1}{2} \mathbf{r}_2 = \begin{bmatrix} 0.2 \\ 0.5 \\ 0.3 \end{bmatrix}$$

User 2 – Teleport set $\mathbf{s}_{u2} = \{1,2,3\}$

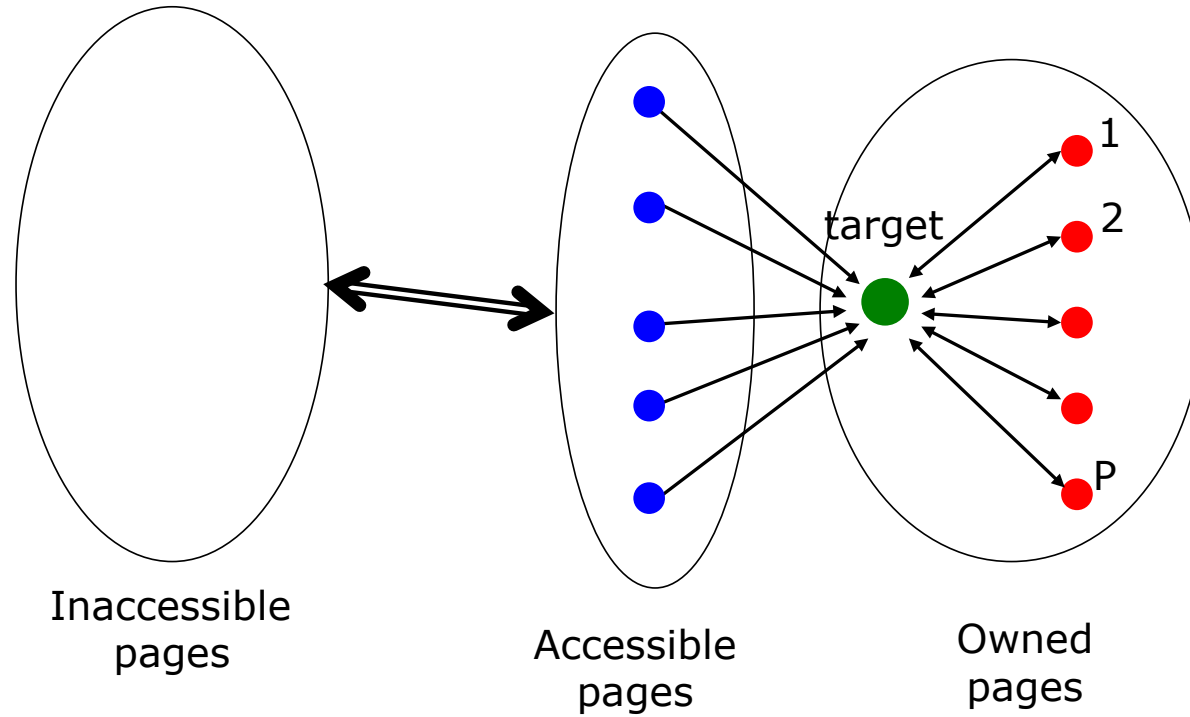
$$\mathbf{p}_{u2} = \frac{1}{3} \mathbf{e}_{\mathbf{s}_{u2}} = \frac{1}{3} \mathbf{e}_1 + \frac{1}{3} \mathbf{e}_2 + \frac{1}{3} \mathbf{e}_3$$

$$\mathbf{r}_{u2} = \frac{1}{3} \mathbf{r}_1 + \frac{1}{3} \mathbf{r}_2 + \frac{1}{3} \mathbf{r}_3 = \begin{bmatrix} 0.2 \\ 0.37 \\ 0.43 \end{bmatrix}$$

Link spam

- The goal of a spammer is to maximize the PageRank of a target page
- Three kinds of web pages for a spammer
 - Inaccessible pages
 - Accessible pages (e.g., spammers can put links in the comments)
 - Owned pages
 - Spammers have full control
 - Include the target page and support pages to boost the rank for the target.
- Ideas to boost PageRank:
 - Post many links from the accessible pages to the target page
 - Construct link farm to get PageRank multiplier effect

Link farms

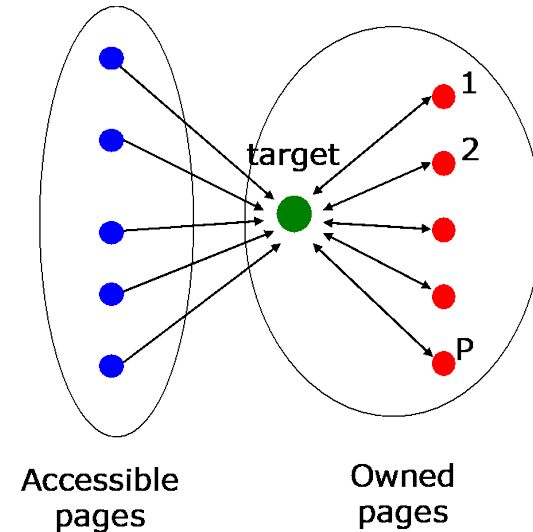


Analysis on link farms

- Assume that without the link farm, the target page has a rank in the form of $y = x + \frac{(1-\beta)}{N}$
 - x : contribution from accessible pages
 - y : PageRank of target page **with link farm**
 - z : PageRank of each support page $z = \frac{\beta y}{P} + \frac{1-\beta}{N}$
 - The rank of the target with link farm:
$$y = x + \beta P z + \frac{1-\beta}{N} \approx \frac{x}{1-\beta^2} + c \frac{P}{N} \quad \text{where } c = \frac{\beta}{1+\beta}$$

Two properties:

- The multiplier $\frac{1}{1-\beta^2} > 1$, boost the contribution of x
- The PageRank grows linearly to the number of support pages.



TrustRank

- A biased PageRank with a teleport set that contains only trustworthy web pages.
- Intuition: a spam page is easy to make a link to a trustworthy page, it is unlikely that trustworthy pages would link to a spam page.
- Pages are divided:
 - Trustworthy pages
 - Fine sites with commentary functionalities
 - Spam pages

Compute TrustRank and consider pages below a threshold as spam? Problem?

Spam Mass

- A complementary view:

What fraction of a page's PageRank comes from **spam** pages?

- Spam mass estimation
 - r_p = PageRank of page p
 - r_p^+ = TrustRank of page p
- The spam mass is:

$$\frac{r_p - r_p^+}{r_p}$$

Pages with high spam mass are spam.